Akhmad Dany – 23031554234

UTS Graf Sains Data 2023 E

Pertanyaan Pendahuluan (Checkpoint)

1. Apa nama database yang digunakan pada tutorial ini?
   Jawab:
   Nama database yang digunakan adalah neo4j, sesuai dengan parameter database_="neo4j" pada kode Python.
2. Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalan 1 instance!
   Jawab:
   Selain database neo4j, kita dapat menggunakan sintaks Cypher untuk mengelola dan mengakses data, antara lain:
   - DDL (Data Definition Language): CREATE, untuk membuat constraint atau index pada database.
   - DML (Data Manipulation Language): CREATE, MERGE, MATCH, SET, dan DELETE untuk membuat, mencari, memperbarui, serta menghapus node dan relasi.

   Untuk mengakses database lain dalam satu instance Neo4j, kita dapat menentukan nama database melalui parameter database_, misalnya database_="nama_database", selama database tersebut tersedia dan pengguna memiliki izin akses.
3. Apa jenis graf pertemanan yang telah di buat di sini?
   Jawab:
   Graf yang dibuat adalah graf berarah karena relasi KNOWS memiliki arah dari satu orang ke orang lainnya. Seperti:
   - Arthur -> Guinevere
   - Arthur -> Lancelot
   - Arthur -> Merlin



Soal UTS

2.1 Wrangling dan EDA

1. Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yang dipilih tiap mahasiswa tidak boleh sama! (akan ditentukan)

   - P1 --> 02/06/2015 sampai 21/08/2015
   - P2 --> 22/08/2015 sampai 11/09/2015

2. Terapkan proses filter ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!

```
Data Hasil filter P1
--------------------
Jumlah baris: 1646494
Tanggal paling awal: 2015-06-02 00:00:01.197000+00:00
Tanggal paling akhir: 2015-08-20 23:59:54.835000+00:00
```

```
Data Hasil filter P2
--------------------
Jumlah baris: 360046
Tanggal paling awal: 2015-08-22 00:00:02.163000+00:00
Tanggal paling akhir: 2015-09-10 23:59:54.555000+00:00
```

```
df_p1.drop(columns=["datetime"]).to_csv("P1_training.csv",index=False)
df_p2.drop(columns=["datetime"]).to_csv("P2_testing.csv",index=False)

print("\nFile berhasil disimpan:")
```

```
✓ 7.1s

File berhasil disimpan:
```

3. Lakukan query untuk mendapatkan komposisi dari subgraph yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!

   Jawab:

   Analisis komposisi subgraph dilakukan untuk mengetahui struktur graf interaksi pengguna dan produk pada periode training (P1), yaitu 2 Juni 2015 sampai 21 Agustus 2015. Analisis ini mencakup jumlah simpul unik Visitor, jumlah simpul unik Item, jumlah keseluruhan relasi interaksi, serta distribusi setiap jenis relasi yang terdapat pada subgraph.

   Proses analisis dilakukan dengan menggunakan query Cypher pada Neo4j yang dieksekusi melalui Python API menggunakan driver.execute_query().

   1. Query Cypher untuk menghitung komposisi subgraph. Query pertama digunakan untuk menghitung jumlah visitor unik, item unik, dan keseluruhan interaksi selama periode P1.

```
query_komposisi = """
MATCH (v:Visitor)-[r]->(i:Item)
WHERE r.timestamp >= $p1_start
  AND r.timestamp < $p1_end
RETURN
    count(DISTINCT v) AS jumlah_visitor,
    count(DISTINCT i) AS jumlah_item,
    count(r) AS jumlah_interaksi
"""
```

- MATCH (v:Visitor)-[r]->(i:Item) digunakan untuk mengambil pola graf yang menghubungkan simpul Visitor dengan simpul Item melalui suatu relasi.
- WHERE digunakan untuk membatasi interaksi berdasarkan timestamp pada periode P1.
- count(DISTINCT v) menghitung jumlah simpul visitor unik yang terlibat dalam interaksi pada periode tersebut.
- count(DISTINCT i) menghitung jumlah simpul item unik yang terlibat dalam interaksi.
- count(r) menghitung jumlah relasi interaksi yang memenuhi kondisi waktu.

2. Hasil Komposisi Graph

	jumlah_visitor	jumlah_item	jumlah_interaksi
0	865191	190020	1664244

Subgraph periode P1 terdiri atas 865.191 visitor unik dan 190.020 item unik yang terhubung melalui 1.664.244 relasi interaksi. Hasil ini menunjukkan bahwa data yang dianalisis merepresentasikan aktivitas pengguna terhadap produk selama periode training. Satu visitor dapat berinteraksi dengan beberapa item, dan satu item dapat berinteraksi dengan beberapa visitor.

3. Query Cypher untuk menghitung jenis relasi

```
query_relasi = """
MATCH (:Visitor)-[r]->(:Item)
WHERE r.timestamp >= $p1_start
  AND r.timestamp < $p1_end
RETURN type(r) AS jenis_interaksi, count(r) AS jumlah
ORDER BY jumlah DESC
"""
```

- MATCH (:Visitor)-[r]->(:Item) mengambil relasi antara visitor dan item tanpa perlu memberikan nama variabel pada kedua simpul.
- WHERE membatasi data berdasarkan timestamp periode P1.
- type(r) mengambil tipe relasi yang digunakan dalam graf.
- count(r) menghitung jumlah relasi untuk setiap tipe interaksi.
- ORDER BY jumlah DESC mengurutkan hasil berdasarkan jumlah relasi dari yang terbesar ke yang terkecil.

4. Hasil distribusi jenis relasi

	jenis_interaksi	jumlah
0	VIEWED	1608441
1	ADDED_TO_CART	42020
2	BOUGHT	13783

Hasil analisis menunjukkan bahwa relasi VIEWED merupakan jenis interaksi yang paling dominan, yaitu sebanyak 1.608.441 interaksi atau sekitar 96,65% dari keseluruhan interaksi. Sementara itu, relasi ADDED_TO_CART berjumlah 42.020 interaksi atau 2,52%, dan relasi BOUGHT berjumlah 13.783 interaksi atau 0,83%.

5. Implementasi Python API

```
records, summary, keys = driver.execute_query(
    query_komposisi,
    p1_start=p1_start,
    p1_end=p1_end,
    database_="neo4j"
)

df_komposisi = pd.DataFrame([record.data() for record in records])
display(df_komposisi)
```

Kode tersebut menggunakan driver.execute_query() untuk mengirim query Cypher ke Neo4j. Hasil yang dikembalikan berupa records kemudian diubah menjadi DataFrame menggunakan Pandas agar jumlah simpul dan relasi dapat ditampilkan dalam bentuk tabel. Dan penggunakan code tersebut juga di gunakan pada menghitung jenis relasi.

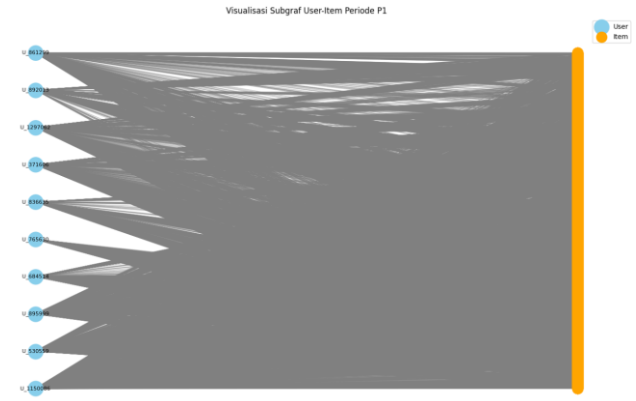
4. Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!

Jawab:

Vector Space Modelling (VSM) dilakukan dengan merepresentasikan setiap User sebagai vektor berdasarkan interaksinya dengan Item pada periode training (P1). Data interaksi diambil dari pasangan unik visitorid dan itemid, kemudian dibentuk menjadi matriks sparse biner menggunakan csr_matrix.

```
Jumlah User: 865191
Jumlah Item: 190020
Shape VSM: (865191, 190020)
Dimensi vektor setiap User: 190020
Jumlah pasangan User-Item: 1298734
Sparsity: 99.999210%

Jumlah User dengan minimal satu interaksi: 865191
Jumlah Item: 190020
Jumlah pasangan User-Item: 1298734
Rata-rata Item per User: 1.5
Interaksi minimum per User: 1
Interaksi maksimum per User: 3328
```



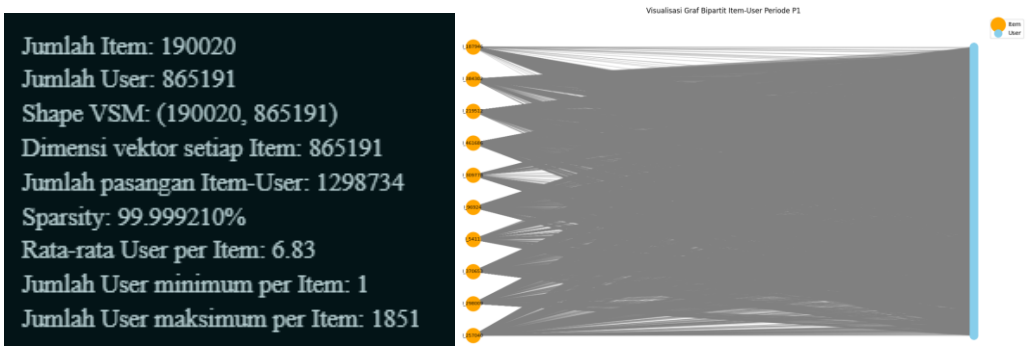
Berdasarkan hasil tersebut, matriks VSM memiliki 865.191 baris yang merepresentasikan User dan 190.020 kolom yang merepresentasikan Item. Setiap User direpresentasikan oleh vektor berdimensi 190.020, dengan nilai 1 menunjukkan adanya interaksi dengan Item dan 0 menunjukkan tidak adanya interaksi yang tercatat pada periode P1.

Nilai sparsity sebesar 99,999210% menunjukkan bahwa sebagian besar elemen matriks bernilai 0. Hal ini terjadi karena setiap User hanya berinteraksi dengan sebagian kecil dari keseluruhan Item. Oleh karena itu, penggunaan matriks sparse lebih efisien dalam penyimpanan data dibandingkan matriks padat (dense matrix).

Visualisasi menggunakan NetworkX menghasilkan graf bipartit dengan 10.051 node dan 14.342 edge. Graf tersebut merupakan visualisasi sebagian data interaksi, bukan keseluruhan graf, karena jumlah User dan Item dalam dataset sangat besar. Node merepresentasikan User dan Item, sedangkan edge menunjukkan hubungan interaksi

antara keduanya. Visualisasi ini membantu menggambarkan pola hubungan User-Item pada periode P1.

5. Lakukan VSM terhadap simpul yang diasumsikan sebagai Item. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!
   Jawab:



   Berdasarkan hasil tersebut, matriks VSM memiliki 190.020 baris yang merepresentasikan Item dan 865.191 kolom yang merepresentasikan User. Dengan demikian, setiap Item direpresentasikan oleh vektor berdimensi 865.191. Nilai 1 menunjukkan bahwa User pernah berinteraksi dengan Item tersebut, sedangkan nilai 0 menunjukkan tidak adanya interaksi yang tercatat pada periode P1.

   Nilai sparsity sebesar 99,999210% menunjukkan bahwa sebagian besar elemen matriks bernilai 0. Hal ini disebabkan setiap Item hanya berinteraksi dengan sebagian kecil dari keseluruhan User. Oleh karena itu, penggunaan matriks sparse lebih efisien dalam penyimpanan data dibandingkan matriks padat (dense matrix).

   Visualisasi menggunakan NetworkX menghasilkan graf bipartit dengan 9.612 node dan 9.956 edge, yang terdiri dari 10 Item terpilih beserta User yang berinteraksi dengannya. Node merepresentasikan Item dan User, sedangkan edge menunjukkan hubungan interaksi di antara keduanya. Visualisasi ini hanya menampilkan sebagian data agar pola hubungan lebih mudah diamati.

6. Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!
   Jawab:
   Kedua VSM tersebut memiliki jumlah pasangan interaksi dan tingkat sparsity yang sama karena dibangun dari data interaksi yang sama, tetapi orientasi baris dan kolomnya berbeda. VSM User memiliki jumlah entitas yang lebih besar, yaitu 865.191 User, sedangkan VSM Item hanya memiliki 190.020 Item.

   Proyeksi Item - Item dipilih sebagai alternatif yang lebih berpotensi efisien karena jumlah simpul yang diproses lebih sedikit dibandingkan proyeksi User–User. Selain itu, proyeksi Item - Item memungkinkan analisis kemiripan antar-Item berdasarkan pola interaksi User yang sama.

7. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan Python
   a. Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi !
      Jawab

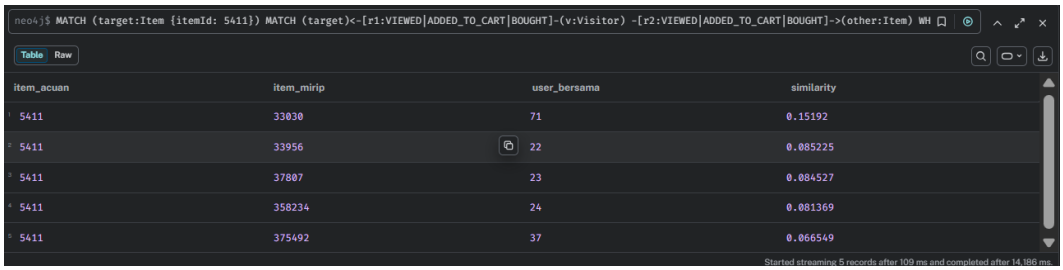
```
MATCH (target:Item {itemId: 5411})
MATCH (target)<-[r1:VIEWED|ADDED_TO_CART|BOUGHT]-(v:Visitor)
      -[r2:VIEWED|ADDED_TO_CART|BOUGHT]->(other:Item)
WHERE other <> target
  AND r1.timestamp >= 1433203200000
  AND r1.timestamp < 1440201600000
  AND r2.timestamp >= 1433203200000
  AND r2.timestamp < 1440201600000

WITH target, other, count(DISTINCT v) AS user_bersama

MATCH (target)<-[rt:VIEWED|ADDED_TO_CART|BOUGHT]-(vt:Visitor)
WHERE rt.timestamp >= 1433203200000
  AND rt.timestamp < 1440201600000
WITH target, other, user_bersama,
     count(DISTINCT vt) AS jumlah_user_target

MATCH (other)<-[ro:VIEWED|ADDED_TO_CART|BOUGHT]-(vo:Visitor)
WHERE ro.timestamp >= 1433203200000
  AND ro.timestamp < 1440201600000
```

   b. Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil querynya!
      Jawab:



      Berdasarkan hasil perhitungan Cosine Similarity, lima Item yang paling mirip dengan Item acuan 5411 telah berhasil diperoleh menggunakan data interaksi periode P1.

      Item 33030 memiliki nilai similarity tertinggi sebesar 0,151920, dengan 71 User yang berinteraksi dengan kedua Item tersebut. Selanjutnya, Item 33956 memiliki similarity 0,085225, Item 37807 sebesar 0,084527, Item 358234 sebesar 0,081369, dan Item 375492 sebesar 0,066549.

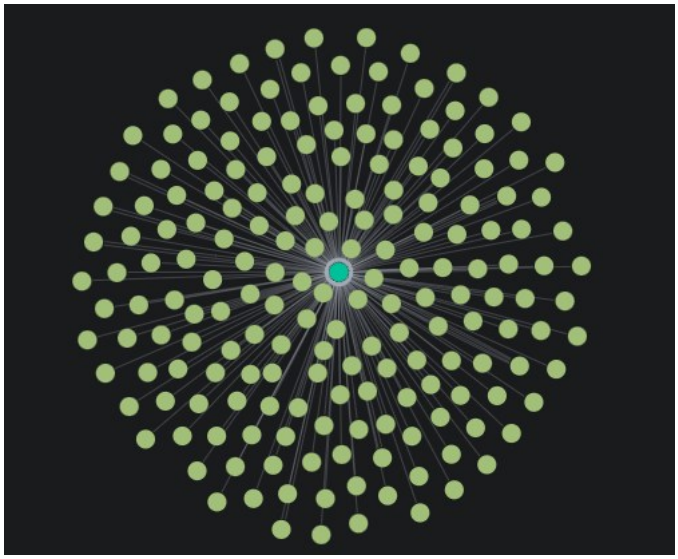
      Hasil ini menunjukkan bahwa Item 33030 memiliki pola interaksi User yang paling mirip dengan Item acuan 5411 dibandingkan empat Item lainnya. Namun, nilai similarity yang relatif rendah menunjukkan bahwa pola interaksi kedua Item tersebut belum terlalu mirip secara keseluruhan.

8. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan Python

   Jawab:

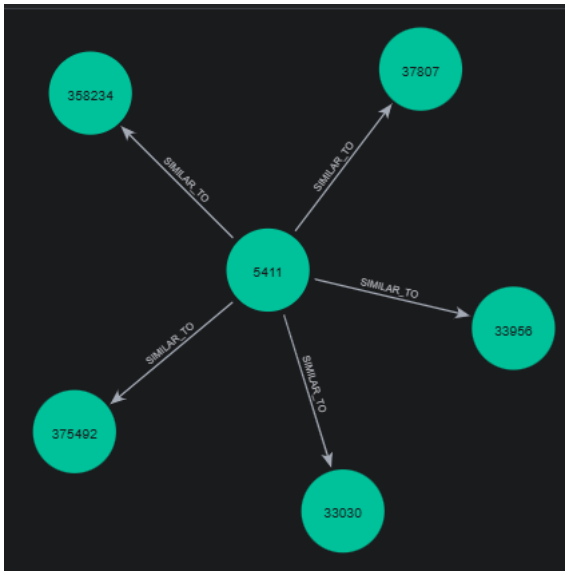
   - Kondisi sebelum perhitungan similarity

```
MATCH (v:Visitor)-[r:VIEWED|ADDED_TO_CART|BOUGHT]->(i:Item)
WHERE i.itemId IN [5411, 33030, 33956, 37807, 358234, 375492]
  AND r.timestamp >= 1433203200000
  AND r.timestamp < 1440201600000
RETURN v, r, i
LIMIT 200
```



   - Kondisi sebelum perhitungan similarity

```
MATCH (target:Item {itemId: 5411})-[s:SIMILAR_TO]->(other:Item)
WHERE s.method = 'Cosine Similarity'
  AND s.period = 'P1'
RETURN target, s, other
ORDER BY s.similarity DESC
```



2.2 Analytics

1. Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
   a. tuliskan skrip Python dalam merekomendasikan KNN-nya!

```python
user_ids = interaksi_p1["visitorid"].unique()
item_ids = interaksi_p1["itemid"].unique()

user_to_idx = {u: i for i, u in enumerate(user_ids)}
item_to_idx = {item: i for i, item in enumerate(item_ids)}

rows = interaksi_p1["itemid"].map(item_to_idx).to_numpy()
cols = interaksi_p1["visitorid"].map(user_to_idx).to_numpy()

vsm_item = csr_matrix(
    (np.ones(len(interaksi_p1)), (rows, cols)),
    shape=(len(item_ids), len(user_ids))
)

print("Shape matriks Item-User:", vsm_item.shape)
print("Jumlah interaksi unik:", vsm_item.nnz)

knn = NearestNeighbors(
    n_neighbors=6,
    metric="cosine",
    algorithm="brute",
    n_jobs=-1
)

knn.fit(vsm_item)

print("Model Item-Based KNN berhasil dilatih.")
print("Parameter k tetangga:", 5)
```

   b. berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!

```
User yang dipilih: 797182

=== TIGA ITEM REKOMENDASI ===
1. Item 318475 | Similarity = 0.176777
2. Item 420628 | Similarity = 0.144338
3. Item 251632 | Similarity = 0.111803

Jumlah rekomendasi yang ditampilkan: 3
```

2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:
   a. Hit@5

   Jawab:

```python
item_relevan = set(interaksi_p2.loc[interaksi_p2["visitorid"] == user_target, "itemid"])

rekomendasi_5 = [item for item, skor in top5]
relevansi_5 = [int(item in item_relevan)for item in rekomendasi_5]

hit_at_5 = int(any(relevansi_5))

print("User yang dievaluasi:", user_target)
print("Top 5 rekomendasi:", rekomendasi_5)
print("Jumlah item relevan pada P2:", len(item_relevan))
print("Relevansi Top 5:", relevansi_5)
print("Hit@5:", hit_at_5)
```

```
User yang dievaluasi: 797182
Top 5 rekomendasi: [np.int64(318475), np.int64(420628), np.int64(251632), np.int64(330794), np.int64(189681)]
Jumlah item relevan pada P2: 1
Relevansi Top 5: [0, 0, 0, 0, 0]
Hit@5: 0
```

b. NDCG@5

Jawab:

```python
dcg_at_5 = sum(rel / np.log2(posisi + 2)for posisi, rel in enumerate(relevansi_5))
jumlah_relevan_top5_maks = min(len(item_relevan), 5)

idcg_at_5 = sum(
    1 / np.log2(posisi + 2)
    for posisi in range(jumlah_relevan_top5_maks)
)

if idcg_at_5 > 0:
    ndcg_at_5 = dcg_at_5 / idcg_at_5
else:
    ndcg_at_5 = np.nan

print("Relevansi Top 5:", relevansi_5)
print(f"DCG@5: {dcg_at_5:.4f}")
print(f"IDCG@5: {idcg_at_5:.4f}")

if np.isnan(ndcg_at_5):
    print("NDCG@5: Tidak dapat dievaluasi (tidak ada item relevan pada P2)")
else:
    print(f"NDCG@5: {ndcg_at_5:.4f}")
```

```
Relevansi Top 5: [0, 0, 0, 0, 0]
DCG@5: 0.0000
IDCG@5: 1.0000
NDCG@5: 0.0000
```